Object-Oriented Programming: Lab Report

Topic: Inheritance, Polymorphism, and Comparable Interface

Name: Vu Duy Quang
Student ID: 202417186
Class: 761926

Reading Assignment: Inheritance & Polymorphism

1. What are the advantages of Polymorphism?

Polymorphism provides several key benefits in software design:

- **Extensibility:** It allows the system to handle new types of objects without modifying existing code. You can add a new class (like `Book` or `CD`) and use it with your existing `Cart` logic immediately.
- **Code Simplicity:** Instead of writing separate methods for every media type (e.g., `playDVD()`, `playCD()`), you can simply call a single `play()` method on a `Playable` reference.
- **Flexibility:** It enables the use of interfaces and abstract classes, decoupling the implementation from the definition.

2. How is Inheritance useful to achieve Polymorphism in Java?

Inheritance is the core prerequisite for polymorphism in Java. By creating a hierarchy where `DVD` and `Book` extend `Media`, you establish a "sub-type" relationship. This allows a superclass reference variable (e.g., `Media m`) to point to any subclass instance. At runtime, Java uses *dynamic method dispatch* to execute the method version specific to the actual object instance, which is only possible because inheritance defined that common parent-child structure.

3. What are the differences between Polymorphism and Inheritance in Java?

- **Inheritance** is a *mechanism* for code reuse where one class inherits fields and methods from another. It defines the **structure** of the classes.
- **Polymorphism** is the *behavior* that results from that structure. It allows a single action (method call) to behave differently depending on the object it is acting upon.
- *In short:* Inheritance is about establishing relationships between classes; Polymorphism is about taking advantage of those relationships to write flexible code.

Comparable Interface Approach

1. What class should implement the Comparable interface?

The `Media` abstract class should implement the `Comparable<Media>` interface. Since the `Cart` and `Store` store a list of `Media` objects, implementing it at the base level ensures that all subclasses can be compared and sorted using standard Java sorting tools.

2. Implementation of the compareTo() method

In the `Media` class, the `compareTo()` method should be implemented based on the "natural" ordering requirement. Typically, this would be by **Title** first, then by **Cost**.

```
@Override
public int compareTo(Media other) {
    int titleCompare = this.title.compareToIgnoreCase(other.title);
    if (titleCompare != 0) {
        return titleCompare;
    }
    return Float.compare(this.cost, other.cost);
}
```

3. Multiple ordering rules with Comparable?

No. A class can only implement `Comparable` once and provide one `compareTo()` method. This defines the "Natural Order" of the objects. If you need two or more different ordering rules (like Title-then-Cost vs. Cost-then-Title), you must use the `Comparator` interface instead, which allows you to define multiple sorting criteria in separate classes or lambda expressions.

4. Different ordering rule for DVDs

To allow DVDs to have a unique rule (Title → Decreasing Length → Cost), you would **Override** the `compareTo()` method specifically inside the `DigitalVideoDisc` class.

```
@Override
public int compareTo(Media other) {
    if (other instanceof DigitalVideoDisc) {
        DigitalVideoDisc otherDVD = (DigitalVideoDisc) other;
        int titleCompare = this.title.compareToIgnoreCase(otherDVD.title);
        if (titleCompare != 0) return titleCompare;

        // Decreasing Length
        int lengthCompare = Integer.compare(otherDVD.length, this.length);
        if (lengthCompare != 0) return lengthCompare;

        return Float.compare(this.cost, otherDVD.cost);
    }
    // Fallback to the generic Media ordering if comparing to non-DVD
    return super.compareTo(other);
}
```